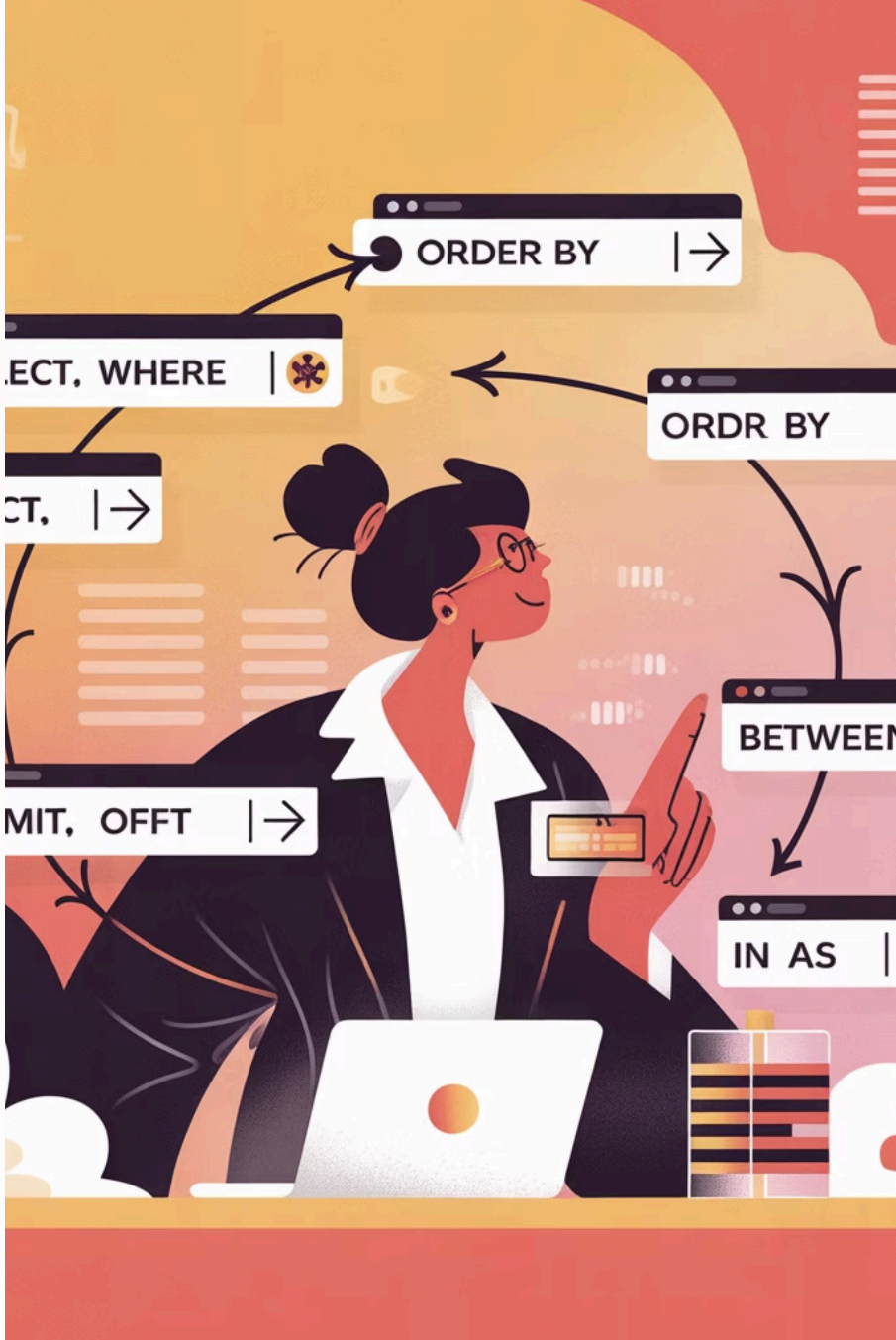
Extracción de Datos con SQL: Queries Básicas

- **SELECT y WHERE:** La Esencia de las Consultas
- **ORDER BY:** Ordenando los Resultados
- **DISTINCT:** Valores Únicos
- **LIMIT y OFFSET:** Controlando la Cantidad
- **BETWEEN:** Selección en un Rango
- **IN:** Coincidencia de Valores Múltiples
- **AS:** Renombrando Columnas



SELECT: El Corazón de la Consulta 📋

Con SELECT podemos extraer datos de una o varias columnas de una tabla.

Sintaxis Básica

```
SELECT columna FROM tabla;
```

Ejemplo (Una Columna)

```
SELECT nombre FROM alumnas;
```

Resultado:

nombre
Ana
Maria
Lucía
...

SELECT: Múltiples Columnas

Seleccionar Varias Columnas

Separa los nombres de las columnas con comas (,).

```
SELECT nombre, apellido FROM alumnas;
```

Resultado:

nombre	apellido
Ana	González
Maria	López

Seleccionar TODAS las Columnas

¡Usa el asterisco (*)!

```
SELECT * FROM alumnas;
```

Resultado: (Muestra la tabla completa)

WHERE: ¡Aplicando Filtros! 🧹

Se utiliza para filtrar los resultados y establecer criterios que los registros deben cumplir.

Sintaxis

```
SELECT columna FROM tabla WHERE columna = condición;
```

Ejemplo

```
SELECT nombre, apellido FROM alumnas WHERE país = "Francia";
```

Resultado:

nombre	apellido
Rocío	García

Nota: ¡Puedes filtrar por columnas que no estás seleccionando!

WHERE: Múltiples Condiciones

AND

Se deben cumplir **todas** las condiciones.

```
... WHERE país = "España" AND
apellido = "Garcia";
```

OR

Se debe cumplir **al menos una** de las condiciones.

```
... WHERE columna = condicion1 OR
columna2 = condicion2;
```

NOT

Niega la condición (busca lo opuesto).

```
... WHERE NOT país = "España";
```

Operadores de Comparación

No solo podemos usar la igualdad (=). ¡Tenemos más opciones!

- `=` : Igual que
- `<>` : Distinto de
- `<` : Menor que
- `>` : Mayor que
- `<=` : Menor o igual que
- `>=` : Mayor o igual que

Ejemplo

```
SELECT ciudad, país FROM alumnas WHERE teléfono <>
674459123;
```

IS NULL y IS NOT NULL

¿Cómo encontramos valores que están vacíos (nulos)?

IS NULL

Selecciona registros donde la columna especificada tiene un valor nulo.

```
SELECT nombre, apellido FROM alumnas WHERE ciudad IS NULL;
```

IS NOT NULL

El opuesto: selecciona registros que SÍ tienen un valor.

```
SELECT nombre, apellido FROM alumnas WHERE ciudad IS NOT NULL;
```

Otras Consultas

¡Continuamos nuestro viaje! Vimos SELECT y WHERE, pero hay más cláusulas importantes que se usan en el día a día.

ORDER BY

Se utiliza para ordenar los resultados de una consulta.

Orden Ascendente (A-Z, 0-9)

```
... ORDER BY columna ASC;
```

Orden Descendente (Z-A, 9-0)

```
... ORDER BY columna DESC;
```

Ejemplo

```
SELECT * FROM alumnas ORDER BY apellido DESC;
```

DISTINCT: Valores Únicos

Se utiliza para seleccionar valores únicos y evitar duplicados en una columna.

Sintaxis

```
SELECT DISTINCT columna FROM nombre_tabla;
```

Ejemplo (Países únicos)

```
SELECT DISTINCT país FROM alumnas;
```

Resultado:

país
España
Francia

Nota: Si se usa con varias columnas, mostrará *combinaciones* únicas.

LIMIT:

Limita el número de filas que se muestran en el resultado. ¡Muy útil con tablas grandes!

Sintaxis

```
SELECT columnas FROM nombre_tabla LIMIT cantidad_filas;
```

Ejemplo (Solo las 2 primeras filas)

```
SELECT * FROM alumnas LIMIT 2;
```

Uso combinado: Se puede (¡y se debe!) usar con ORDER BY para obtener los "Top 10" o "Últimos 5".

OFFSET:

Se usa *junto* con LIMIT para descartar (saltar) los primeros 'N' resultados.

Sintaxis

```
SELECT columnas FROM nombre_tabla LIMIT cantidad OFFSET inicio;
```

Ejemplo (Saltar las 3 primeras, mostrar 10)

(Imaginando una tabla empleadas con salarios)

```
SELECT nombre, apellido, salario FROM empleadas ORDER BY salario DESC LIMIT 10 OFFSET 3;
```

BETWEEN: Selección en un Rango

Selecciona registros cuyos valores están dentro de un rango (¡ambos valores incluidos!).

Sintaxis

```
... WHERE columna BETWEEN valor_inicio AND valor_fin;
```

Ejemplo (IDs del 3 al 5)

```
SELECT nombre, apellido FROM alumnas WHERE id_alumna
BETWEEN 3 AND 5;
```

Resultado: (Lucía, Elena, Rocío)

IN: Coincidencia de Valores Múltiples

Compara un valor con una lista de posibles valores. ¡Es como usar muchos OR de forma más limpia!

Sintaxis

```
... WHERE columna IN (valor1, valor2, valor3);
```

Ejemplo (Solo ciudades específicas)

```
SELECT nombre, apellido FROM alumnas WHERE Ciudad IN
('Madrid', 'Valencia', 'Barcelona');
```

Resultado: (Ana, Maria, Lucía)

AS: Renombrando Columnas y Tablas

Se usa para dar alias (nombres alternativos) para que los resultados sean más claros y descriptivos.

Sintaxis (Alias de Columna)

```
SELECT columna AS alias_columna FROM
nombre_tabla;
```

Sintaxis (Alias de Tabla)

```
SELECT columna
FROM nombre_tabla
AS alias_tabla;
```

Ejemplo

```
SELECT nombre AS Name,
apellido AS Surname
FROM alumnas AS Students;
```

¡Importante! El Orden de Escritura

En SQL, ¡el orden importa! No podemos escribir las cláusulas de cualquier forma. 🤔

1	**SELECT (y AS)**
2	**FROM**
3	**WHERE (con BETWEEN, IN, NOT, etc.)**
4	**ORDER BY**
5	**LIMIT (y OFFSET)**

Nota: ¡Más adelante veremos JOIN, GROUP BY y HAVING y dónde encajan en esta lista!

Resumen

1	**SELECT** Añade el nombre de la columna o columnas que queremos extraer.
2	**FROM** Indica la tabla de la que extraemos.
3	**WHERE** Pone condiciones (filtros) para extraer los datos.
4	**IS NULL / NOT NULL** Lo usamos en el WHERE para obtener los registros nulos o no nulos.
5	**ORDER BY** Ordena los registros en base a una columna (ASC o DESC).
1	**DISTINCT** Devuelve valores únicos de una columna.
2	**LIMIT** Limita el número de registros (ej. LIMIT 10).
3	**OFFSET** Omite los primeros 'X' registros (se usa *siempre* con LIMIT).
4	**BETWEEN** Para rangos (ej. WHERE precio BETWEEN 50 AND 90).
5	**IN** Para listas (ej. WHERE pais IN ('USA', 'Francia')).
6	**NOT IN** Productos que no sean de esa lista.
7	**AS** Crea un alias (ej. quantity_in_stock * buy_price AS total_stock).